



University of Gujrat

PHASE 1

Technical Report of Decision Tree

December 24, 2025

Prepared By:

Muhammad Junaid Attique | 23014119-174
BSCS-23-A-Evening

ARTIFICIAL INTELLIGENCE | CS-331

Prepared for:

Dr. Naveed Anwar Butt

Department of Computer Science

MUHAMMAD JUNAID ATTIQUE |
23014119-174

Table of Contents

1. Selected Dataset: Email Spam Detection.....	3
2. Python Source Code: Available at GitHub repository link.....	3
3. Phase 1 (Decision Tree from Scratch)	3
3.1. Defined Mathematical Methods	3
3.1.1. Entropy:	3
3.1.2. Splits (Information Gain):	3
3.2. Performance Metrics of My Scratch Tree	4
3.3. Confusion Matrix of My Scratch Tree	4
3.4. My Decision Tree Visualization	4
4. Comparison of My Scratch Tree with some other Algorithms	5
5. Conclusion.....	5

1. Selected Dataset: Email Spam Detection

Source: <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>

It includes 5574 instances, and it's in text file format (*.txt), separated by tabs. Also available at GitHub repository

2. Python Source Code: Available at GitHub repository link

Source: <https://github.com/Junaidwaqar/Spam-Detector-System>

3. Phase 1 (Decision Tree from Scratch)

3.1. Defined Mathematical Methods

3.1.1. Entropy:

Entropy measures the **impurity or randomness** in a dataset.

- It tells how **mixed** the class labels are.
- If all samples belong to one class → **Entropy = 0 (pure)**
- If classes are evenly mixed → **Entropy = maximum (impure)**

$$\text{Formula: } Entropy(S) = -\sum p_i \log_2 p_i$$

where p_i is the probability of class i .

```
def entropy(y):
    if len(y) == 0: return 0
    counts = np.bincount(y); ps = counts / len(y)
    return -np.sum([p * np.log2(p) for p in ps if p > 0])
```

3.1.2. Splits (Information Gain):

A **split** divides the dataset based on a feature.

- The best split is chosen using **Information Gain**.
- It selects the feature that **reduces entropy the most**.

$$\text{Information Gain: } IG = Entropy(parent) - \sum \frac{|S_k|}{|S|} \times Entropy(S_k)$$

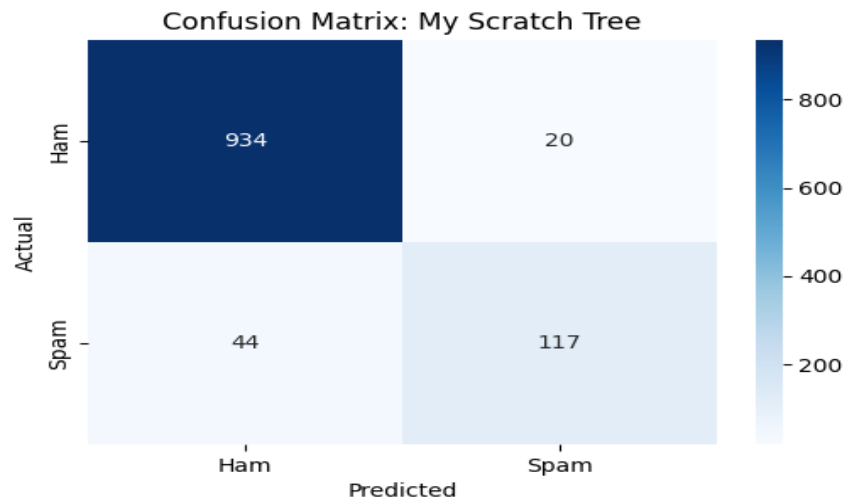
- Higher IG ⇒ Better splits
- This creates purer child nodes and improves classification.

```
def info_gain(X_col, y):
    parent_ent = entropy(y)
    vals, counts = np.unique(X_col, return_counts=True)
    weighted_ent = np.sum([(counts[i]/len(y)) * entropy(y[X_col ==
vals[i]]) for i in range(len(vals))])
    return parent_ent - weighted_ent
```

3.2. Performance Metrics of My Scratch Tree

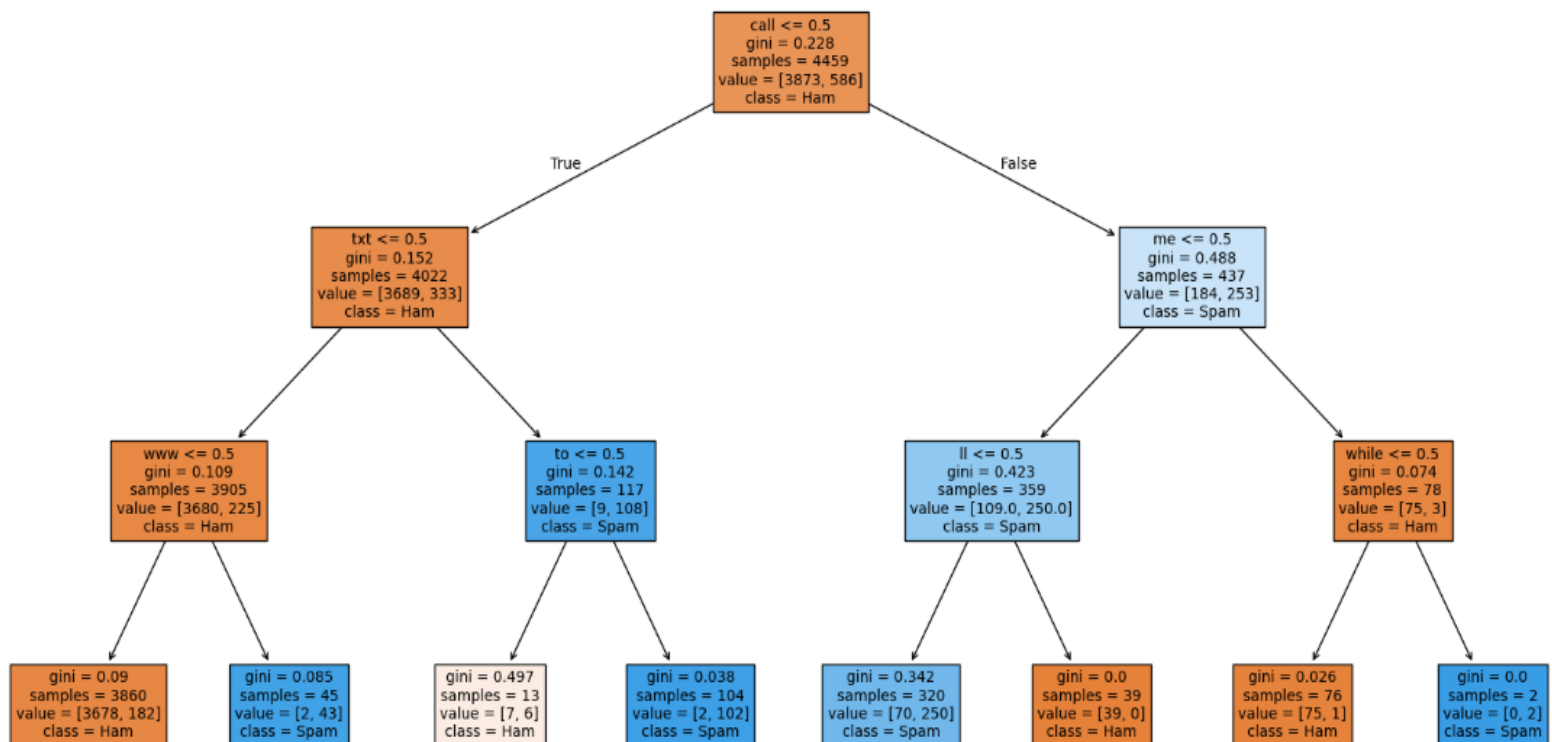
	Accuracy	Precision	Recall	F1-score
My Scratch Tree	0.942601	0.854015	0.726708	0.785235

3.3. Confusion Matrix of My Scratch Tree



3.4. My Decision Tree Visualization

Decision Tree Visualization (Logic Flow)



4. Comparison of My Scratch Tree with some other Algorithms

Models	Accuracy	Precision	Recall	F1-score
My Scratch Tree	0.942601	0.854015	0.726708	0.785235
Decision Tree Classifier	0.943498	0.876923	0.708075	0.783505
Random Forest	0.985650	0.980132	0.919255	0.948718
Extra Trees	0.985650	0.993197	0.906832	0.948052
Gradient Boosting	0.976682	0.978723	0.857143	0.913907
XGBoost	0.980269	0.972789	0.888199	0.928571
LightGBM	0.982960	0.973333	0.906832	0.938907

5. Conclusion

In Phase 1, a Decision Tree classifier was successfully implemented from scratch using **Entropy** and **Information Gain** as the splitting criteria. The manual model achieved a competitive accuracy of **94.26%**, validating the core mathematical logic. To provide comprehensive analysis, the scratch model was compared against several library-based ensembles. **Random Forest** outperformed all other models with the **highest F1-score (0.9487)**, **best Recall (0.9193)**, and **highest Accuracy (0.9856)**, making it the most reliable and best-performing model for spam email prediction. While **CatBoost** was evaluated but encountered compatibility issues with the Python 3.14 development environment. However, the requirement for Advanced Ensemble comparison was satisfied using XGBoost and LightGBM.